

Hosting Tree Speculation on a Recurrent-Hybrid Model: Verifier Design, Audit Methodology, and Serving Evidence

Anonymous Author(s)

ABSTRACT

Tree speculative decoding verifies many candidate continuations per step, but a recurrent-hybrid language model can host it only if every accepted path leaves consistent recurrent state, convolution history, and attention cache behind. We present TreeHost, a vLLM serving design in which a shared tree descriptor drives path-parallel recurrent verification, device-resident acceptance, and coordinated publication of the selected path into native running state. We also contribute the measurement and audit procedures behind the evidence: a pooled completed-request decode-rate estimator with explicit exclusions, frozen recorded-request corpora with a disjoint-task generalization check, a frozen native-envelope numerical rule with planted state faults, a captured-row and Monte-Carlo sampler audit with planted faults, and an identical-prompt output-length comparability check. On Qwen3.8-27B NVFP4 on an NVIDIA DGX Spark, TreeHost verifies 27-candidate trees at 28.13 and 30.76 pooled tokens/s on two corpora, 7.7% and 11.8% above the same stack’s five-token chain speculation, and 5.3% below and 0.9% above a tuned chain-speculation stack on a second serving system. Within one engine, hosting 32-row tree verification costs 19–20% more per step than six-row chain verification and returns 28–34% more accepted tokens per step. Across 126 held-out continuation cycles the verifier’s state and logits stay within twice the native process-to-process spread while all ten planted faults are detected; the deployed sampler, audited on 2,480 captured rows, is exact for the probabilities it receives up to a top- k tie convention, and the audit found and led to the repair of a penalty-history defect before measurement. A ten-task coding-agent study is reported descriptively.

CCS CONCEPTS

• **Software and its engineering** → **Software performance**; • **General and reference** → **Measurement**; • **Computing methodologies** → *Natural language processing*.

KEYWORDS

speculative decoding, hybrid language models, Gated DeltaNet, LLM serving, measurement methodology, numerical reproducibility

1 INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel; its sampling guarantee assumes that verification supplies the intended target probabilities, and tree speculation widens the candidate set without relaxing that assumption [3, 15, 18]. Recurrent-hybrid models such as Qwen3-Next, which interleave Gated DeltaNet (GDN) layers with softmax

attention, complicate the verifier: a candidate must inherit the recurrent state, convolution history, and attention cache of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [30, 31]. A verifier can compute plausible candidate logits and still publish the wrong state.

This paper makes three contributions. *First*, a verifier design that hosts 27-candidate trees on a hybrid model inside a production serving engine: branch paths are scanned in parallel from shared boundary states, acceptance runs on the device, and one accepted path is replayed into the native recurrent, convolution, and attention state through a shared logical-to-physical descriptor. A chain verifier cannot host tree drafters at all; this design is the substrate they need, and its narrow distinction from the closest designs is that it commits through the engine’s own per-layer recurrent update, which costs a replay pass per step and buys bitwise-native published state (Section 2). *Second*, the measurement and audit procedures for speculative serving on a hybrid model, where decode rates, numerical agreement, and sampling correctness are each easy to mismeasure: a pooled completed-request decode-rate estimator with explicit exclusions; two frozen recorded-request corpora with a disjoint-task generalization check; a native-envelope numerical rule frozen before data collection, with ten planted state faults; a captured-row and Monte-Carlo audit of the acceptance walk with planted sampler faults; and an identical-prompt output-length check. The procedures are reusable; the state surfaces, thresholds, and instrumentation are specific to this model and deployment. Their value is demonstrated by a penalty-history defect the sampler audit found in the deployed system, which was repaired before the measurements reported here. *Third*, evidence under that methodology on Qwen3.8-27B NVFP4 on an NVIDIA DGX Spark (GB10): TreeHost exceeds the same stack’s five-token chain speculation by 7.7% and 11.8% pooled tokens/s on the two corpora and lands 5.3% below and 0.9% above a tuned chain-speculation stack on a second engine; within one engine the tree step costs 19–20% more than the chain step and accepts 28–34% more tokens per step, which is the measured cost of hosting tree verification; 126 held-out continuation cycles stay within twice the native spread with every planted fault detected; the deployed sampler, audited on 2,480 captured rows, applies correct penalty histories and is exact for the probabilities it receives up to a top- k tie convention. A ten-task SWE-bench Verified coding-agent study is reported descriptively.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, so performance depends on completed work as well as token production. We therefore report the model, serving route, task cohort, and measurement definition together, and we separate recorded-request replay, which fixes the prompts, from live agent runs, which do not. Section 2 positions the design against the closest mechanisms; Section 3 describes it; Section 4 defines the methodology; Section 5

reports the evidence; Section 6 bounds it. Raw records, reducers, and the serving patches are in the anonymized artifact [1].

2 BACKGROUND AND CLOSEST DESIGNS

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [3, 15]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, OPT-Tree, EAGLE-3, and DFlash study tree construction and drafters [4, 5, 16–18, 26]; these drafting mechanisms are distinct from the verifier and state-publication choices studied here. PagedAttention and SGLang supply the scheduling and memory substrate, Marconi addresses prefix caching for hybrid models, and hybrid serving manages recurrent-state pools alongside KV storage [14, 21, 23, 24, 36]. Flash-Decoding, FlashInfer, DeFT, and FastTree establish split-KV execution and tree-aware KV reuse [7, 22, 34, 35]; our modified FlashAttention-2 decode kernel applies grouped split-K execution under the tree’s visibility and slot mapping [6].

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [31, 32]. Snakes and Ladders and STree use activation replay to recover state-space continuation state; SpecMamba and component-aware self-speculation study hybrid speculation further [2, 29, 30, 37]. These works motivate an explicit state lineage; they do not make every recurrent update or numerical implementation interchangeable.

Six designs verify GDN trees or share recurrent state across candidates (Table 1). Weaver verifies with an ancestor-masked triangular solve and replays the selected path at commit time [19]; SpecLA keeps value tiles of recurrent state on chip within a chain, schedules ready chains in parallel, and buffers accepted factors for a later fused update [28]; Bole evaluates a tree-structured finite-Neumann closed form, stores compact factors, reconstructs the accepted state, and integrates with SGLang [27]; TreeWY solves a triangular system and fuses reconstruction of the previous accepted state with the next verification [9, 10]; ReplaySSM caches inputs and defers state materialization [8]; OneLA shares a prompt-derived state across recommendation beams [33]. Read-only verification followed by accepted-path replay and parallel chain scheduling are therefore direct prior art. TreeHost differs in continuation policy: it replays recorded operands through the serving engine’s native per-layer GDN updates immediately after acceptance and publishes convolution and attention state through the same descriptor, so each iteration returns to the native state representation that prefix caching and serving graphs already use. SpecLA also exchanges boundary states between chains, and Bole also commits accepted state on the device inside a production engine, so the distinction is narrow: the replay pass is explicit per-step work (Section 5.3, Appendix F), and what it buys is published recurrent state bitwise equal to the native chain’s and prefix-cache resumption through unchanged engine interfaces (Sections 5.4 and 5.5). Published speed ratios from different models and workloads are not combined into a ranking.

3 VERIFIER DESIGN

Table 1: Recurrent verification and continuation policies. Mechanism comparison only; no performance ranking.

Method	Verification work	Continuation state
Weaver [19]	Ancestor-masked triangular solve	Short selected-path recurrence
SpecLA [28]	Resident state tiles; parallel chains	Buffered factors; later fused update
Bole [27]	Finite-Neumann correction solve	Compact factors; state reconstruction
TreeWY [10]	Triangular solve in fused kernel	Prior-path reconstruction with next verify
ReplaySSM [8]	Corrected inputs; cached transitions	Deferred state update
OneLA [33]	Shared-base projections across beams	Compact per-beam transition records
TreeHost	Resident state tiles; parallel paths	Native replay; coordinated convolution and KV publication

3.1 State boundary and shared descriptor

Let m be a materialized token prefix, the tokens already consumed by the target’s forward computation, with persistent state $\mathcal{S}(m) = (R(m), C(m), K(m))$: recurrent matrices R , convolution histories C , and attention KV entries K . The logical emitted prefix may include a newly sampled token b not yet consumed, so the continuation boundary is $(\mathcal{S}(m), b)$ rather than a claim that $\mathcal{S}(mb)$ exists. A tree descriptor holds candidate token IDs, parent indices, positions, attention visibility, and the map between logical nodes and physical slots. For one GDN head with state $R \in \mathbb{R}^{d_v \times d_k}$, decay α_i , and write strength β_i , a node applies [31]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (1)$$

where $\pi(i)$ is the parent, which need not be the previous packed row. Causal convolution likewise needs path predecessors, and attention visibility must select the same logical ancestry; a correct attention mask cannot repair a wrong recurrent parent.

Figure 2 shows the cycle. Native multi-token prediction (MTP) supplies a principal chain and runner-up candidates; suffix prediction extends selected paths within a bounded capacity [12, 20, 25]. The verifier derives attention visibility, physical packing, path scheduling, and the acceptance walk from one logical topology; padding keeps execution shapes reusable while remaining invisible to attention and selection. The attention layout places the principal spine first in the physical suffix and applies the same permutation to KV write slots and ancestry-mask key columns, retaining logical query-row and position order; acceptance stays in logical-node space and publication maps selected nodes back to physical slots, so a layout optimization cannot silently change which entries a branch reads. Algorithm 1 states the contract: verification, selection, and publication all consume the same materialized path A , and a newly sampled token b^* can be emitted while remaining pending. History-dependent logits processing (penalties) builds each verification row’s history from its own root-to-node path (Section 4.4). Figure 1 summarizes the interface: all three state surfaces describe the same materialized prefix, and a pending token is created by the next forward.

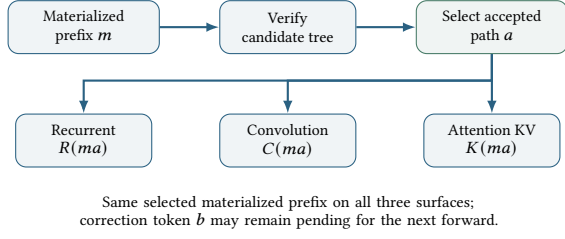


Figure 1: The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix; a newly sampled correction or bonus token can remain pending until the next forward.

Algorithm 1 Tree verification and native continuation

Input: Native continuation state; candidate tree T ; proposal probabilities q
Output: Emitted tokens; updated native state; next drafter input; any pending token

- 1: Map T to physical slots, positions, and ancestor masks.
- 2: $p \leftarrow$ target probabilities from path-local GDN, convolution, and attention computation; retain candidate hidden states, rounded replay operands, convolution histories, and temporary KV entries.
- 3: $(A, b^*) \leftarrow$ accepted materialized path and pending token from sampling with p and q .
- 4: **for** each GDN layer **do**
- 5: Replay recorded operands along A into the native recurrent state; gather the convolution history for A into the native running row.
- 6: **end for**
- 7: Remap the accepted KV entries with the same logical-to-physical slot map; select the matching hidden-state row for the next drafter input.
- 8: Advance positions along A ; keep b^* pending until its target forward.

3.2 Path-parallel scan and accepted-path replay

The GDN verifier decomposes the tree into dependent path groups, a scheduling family also used by SpecLA [28]. A first path starts from the request’s pre-tree state and exports the cut states that downstream paths need; a second level evaluates independent branch paths from those cut states. Each GPU program covers one path, head, and value tile and keeps the full key dimension in its working tile, so sequential updates reuse the tile through the path loop instead of loading a full recurrent state per node. Cut states and operand rings are temporary within a step; the durable continuation lives in the request-owned native running rows, and prefix-cache snapshots use that native layout. The two levels run as separate launches and exchange only boundary states. If a program exports N_c cut states of value-tile width B_v and key dimension d_k , their fp32 payload is $4N_cB_vd_k$ bytes; the reusable scratch allocation reserves a full slot per node, so allocated bytes differ from state traffic.

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. Accepted-path replay uses the engine’s native fused GDN update with the recorded accepted operands, one native call per layer in a captured graph, publishing in place to the fp32 state bank. Normalization, beta conversion, gate evaluation, and output casts can differ between the two implementations, so their finite-precision agreement

is a measured property (Section 5.4), not a consequence of the shared real-arithmetic recurrence. Replay carries one state tile through a bounded path loop, $O(B_vd_k)$ in tree size; recorded operands and candidate outputs scale with the verification shape. Figure 3 shows the state lifetime; Algorithm 2 in Appendix D gives the full procedure. Path decomposition caches state only where downstream paths branch and reuses it across independent work; recomputing ancestry would remove exports at the cost of repeated updates, and exporting every candidate state would trade more writes for less recomputation. The two-level schedule is one point in that space, and neither a shorter logical path nor fewer state writes alone establishes a faster complete request, which is why Section 4 measures complete cycles. The replay pass is the price of this design; Section 5.3 reports its share of the serving step and Appendix F its isolated component cost.

3.3 Implementation changes around the verifier

Three representations must agree: logical tree nodes, temporary verification storage, and the native state used by the next forward. Table 2 lists the five implementation changes that carry one accepted path across them and the constraint that makes each nontrivial.

One device-selected path drives continuation. Selecting output tokens does not update a hybrid model’s state: the recurrent matrix must contain the accepted updates, convolution must retain the accepted history, attention must reference the accepted KV entries, and the drafter must receive the matching hidden row. Reconstructing these choices on the host inserts path-list preparation and synchronization between acceptance and publication. TreeHost instead emits fixed-shape device products (accepted node indices, path lengths, emitted tokens, the selected final row) that feed captured native replay, convolution gathering, KV remapping, and drafter-row selection. Recorded operands are staged for the native per-layer updates in one captured committer graph; emitted and materialized tokens stay distinct, and padded path entries never advance state. Serving, drafter, and committer stages are captured as separate graphs.

One slot map governs attention and publication. An ancestry mask specifies visible keys but not their physical placement or reduction groups; interleaving masked branch columns with spine keys changes the online-softmax reduction layout even when visibility is unchanged, which matters for finite-precision execution [11]. TreeHost places the principal spine contiguously, applies the same permutation to KV write slots and mask key columns, keeps acceptance in logical-node space, and translates the selected nodes through that map at publication. The patched FlashAttention-2 decode kernel adds grouped query heads over shared staged KV tiles and split-K context partitions, established techniques [22, 34] applied here under the tree’s visibility and slot mapping; the component tests in Section 5.4 measure its repeatability and reduction-order error, and the isolated effect of spine placement remains an ablation question.

Precomputed branch histories with preserved arithmetic. Static gather indices derived from the topology, windows gathered from the prior convolution history and candidate inputs, and fused ordered taps replace per-node history construction repeated across

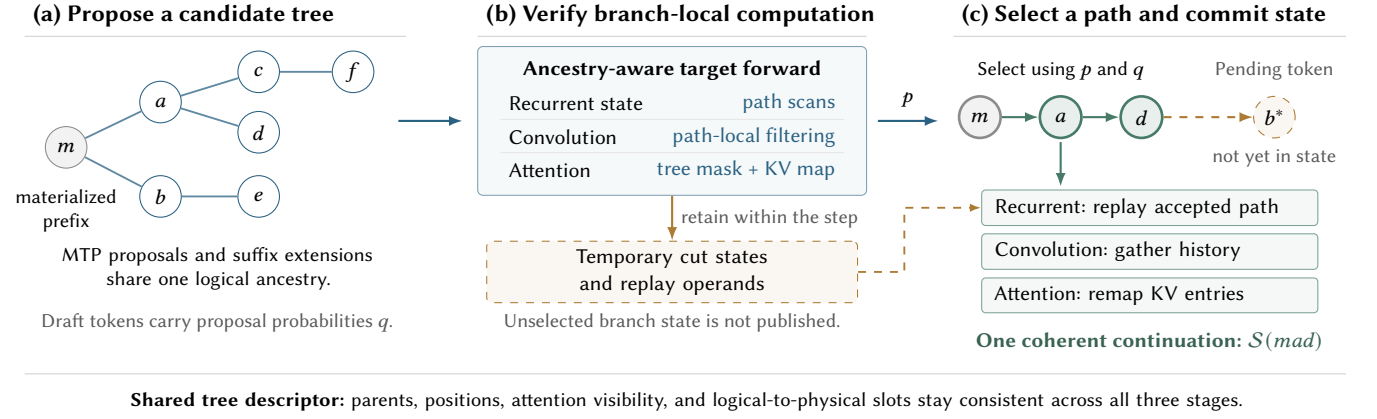


Figure 2: Tree verification and accepted-path commitment. (a) Drafting provides a candidate tree and proposal probabilities q . (b) Branch-local computation produces target probabilities p and retains temporary state. (c) Sampling selects a materialized path (here a, d); recurrent replay, convolution gathering, and KV remapping publish $S(mad)$. The newly sampled token b^* remains pending. One descriptor keeps logical ancestry and physical storage aligned.

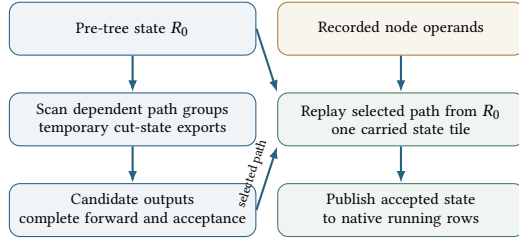


Figure 3: State lifetime in the path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

layers. Tap order and cast boundaries affect the values recorded for replay, and unused history positions must keep their padding, so the fused implementation retains the ordered operations and conversion points while replacing only the data movement; each layer still records and replays its own operands.

Fused draft selection that preserves proposals. At each MTP depth one full-vocabulary logits tensor serves the principal token and sibling candidates, and a fused kernel writes the spine token and three ordered candidate indices into reusable graph buffers. Reference argmax and top- k do not share tie behavior, so taking the first top- k result as the spine token would change the proposal tree; the fused selection preserves both reference outputs, including duplicate proposals where the reference produces them, and exact-output and captured-replay tests check it (Section 5.4).

4 MEASUREMENT AND AUDIT METHODOLOGY

4.1 Platform, arms, and workloads

All experiments use Qwen3.8-27B on one NVIDIA DGX Spark (GB10) with a ModelOpt mixed-precision checkpoint: NVFP4 target

MLP weights and vocabulary projection, FP8 attention projections, BF16 activations and KV storage, FP32 recurrent state, and the checkpoint’s one-layer BF16 MTP module as the only drafter in every arm. Three serving arms share this checkpoint: TreeHost and native five-token chain MTP on the same vLLM build, and SGLang EAGLE on a separate stack that invokes the same MTP module. The evaluated tree has 27 draft candidates and one root padded to 32 verification rows: five MTP passes supply 15 candidates and suffix prediction adds a six-token principal tail and rescue branches of four and two, for a maximum depth of eleven. Attention groups query heads in pairs over four context partitions; the model has 48 GDN and 16 attention layers. Sampling is temperature 0.6, top- p 0.95, top- k 20, min- p 0, presence penalty 1.0, with no per-request seed. Appendix A lists the serving settings. Three workloads are used: (i) a live coding-agent study, Qwen Code 0.19.4 on ten explicitly identified SWE-bench Verified Astropy tasks [13], once per task and arm, with a 24,000-token response cap and a 9,000-second task budget, where the benchmark evaluator tests nonempty patches and empty patches count as unresolved; (ii) recorded-request replay on two frozen corpora; and (iii) full-model continuation and component diagnostics.

4.2 Decode-rate estimator and replay corpora

For completed request r , let n_r be the output-token count, ℓ_r the end-to-end latency, and f_r the time to first output. We report the pooled decode rate

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)}, \quad (2)$$

over the same completed requests, so that the numerator counts output intervals and the denominator excludes first-output latency and between-request agent or tool work. Pooling matched token and time totals across runs weights long requests by their duration; the estimator measures token production over accumulated request time, not aggregate wall-clock throughput, and a change in it is not

Table 2: Implementation changes around the path verifier and the coordination or numerical constraint that makes each nontrivial. Workload rates measure their combined execution.

Starting work or constraint	TreeHost change	What must remain consistent
Host path-list construction between acceptance and publication	Device-selected path feeds captured native replay, convolution, KV and drafter selection	All consumers select the same materialized prefix; pending and padded tokens do not advance state
Logical ancestry does not determine physical KV placement	Spine-contiguous slots with coordinated writes, mask columns and accepted-path remap	Logical query order, physical source slots and native destination positions agree
Per-node convolution histories and repeated layer preparation	Static ancestry gathers, fused ordered taps and per-layer operand buffers	Tap order, cast boundaries, history padding and layer identity
Separate score preparation and argmax/top- k operations	One shared logits tensor and fused selection into graph buffers	Both reference tie conventions, candidate order and duplicate behavior
Repeated KV staging and limited decode parallelism	Grouped query heads and split-K in the patched tree-attention kernel	Tree visibility and slot mapping across partitions; characterized reduction error

a task-completion speedup. For the live study the counts come from server metric counters bracketing each arm, checked for continuity, resets, and outstanding requests at task boundaries; for replay they come from the client stream and server completion counts.

The first replay corpus holds 43 requests recorded from two Astropy task conversations (prompt lengths 23,024–68,557 tokens); each configuration receives the same saved messages and tool definitions sequentially with one active request, a 1,024-token response cap, and the sampling settings above, and generated tool calls are not executed. vLLM arms share image, tokenizer, chat template, prefix caching, chunked prefill, and a 131,072-token context. For the cross-stack comparison the complete input token sequence is captured from vLLM’s tokenizer and passed as IDs to SGLang, so all 43 inputs are token-identical. The second, disjoint corpus selects 43 of 61 requests captured from native-MTP agent executions on four further tasks across five conversations, allocated proportionally and spaced through capture order, with its manifest frozen before any replay and no message overlap with the first corpus; SGLang uses its chat endpoint there, so inputs match as messages but not as serialized IDs; it too is replayed three times. Replay timing starts at the first nonempty streamed delta (first streamed completion-count increase for SGLang’s token-ID endpoint) and ends at stream completion. Each principal vLLM arm is replayed three times per corpus; an exploratory native MTP sweep over one, three, five, and seven draft tokens and SGLang step/capacity settings has unequal repetition.

4.3 Numerical agreement rule and planted faults

Component tests compare the tree verifier and the native chain on synthetic operand sets with the deployed tree geometry (32 physical rows, 28 accepted publication paths, 48 recurrent instances per set) against a float64 recurrence, requiring both root-mean-square and maximum absolute error to satisfy $E_{\text{tree}} \leq 1.10E_{\text{native}} + 2^{-24}M_{64} + 2^{-149}$, where M_{64} is the reference’s corresponding magnitude; each block runs twice in two fresh processes, and planted controls (sibling substitution, swapped state rings, stale publication metadata, off-path poisoning, zero replay) must be rejected.

The full-model comparison replays harvested natural accepted paths over six fresh prefixes of 23,000–46,000 tokens from four task conversations disjoint from both replay corpora, in five fresh serving processes: a sequential reference A , an independent process

B , a native cache-capacity control V matched to the candidate’s KV allocation, a native packed-GDN-kernel variant P , and Tree-Host. Each prefix supplies three seven-cycle schedules (cold with a common starting-state import, a cache-priming repeat, and a cache-resumed case without import), 126 cycles in total including a final root-only cycle that consumes the last pending token; every candidate forward is checked against all 32 recorded tree-input rows. For state or logit surface s at cycle c the reference envelope is $R_s(c) = \max\{F_s, d_s(B, A; c), d_s(V, A; c), d_s(P, A; c)\}$, with state distance the maximum per-layer relative L_2 error (floor $F_s = 0.02$) and next-token KL divergence (floor 0.15 nats); the candidate must satisfy $d_s \leq 2R_s$ on every surface and cycle, and a greedy mismatch is tolerated only when A ’s top-two logit margin lies within $\max(0.25, 2 \max_{B,V,P} \|\Delta \text{logit}\|_\infty)$ or the native controls are themselves categorically unstable there, with at most 10% of cells native-unstable. Passing establishes the declared engineering criterion, candidate deviation no larger than twice the native process-to-process spread, not small absolute deviation and not sampling equivalence; the KL floor makes that bound at least 0.30 nats, so the logit surface discriminates planted faults rather than fine differences. The rule and constants were calibrated on an earlier collection and frozen before the fresh prefixes were evaluated. Ten planted faults, five types on two prefixes each (omitted convolution gathering, omitted recurrent replay, incorrect KV remapping, stale root input, sibling-state publication), must each exceed the base case’s bound on the designated surface; cache-resumed cases must start at least 90% cached; one candidate case is repeated in-process. Appendix C details the protocol.

4.4 Sampler audit

The sampling rule is exact only when the supplied probabilities are correct, which for history-dependent processing means that a target row’s penalty history is the path through its parent and a leaf continuation row’s history includes the leaf. The audit has two parts. A record-only runtime wrapper captures, for 40 tree steps on four requests, the token set whose logits the processors changed in each of 2,480 target and continuation rows, and compares it with the per-path history reconstructed from the tree’s parent table and with the flattened chain history a chain combiner would build. Then the deployed acceptance walk, whose CPU floating-point oracle the CUDA path matches byte for byte, is sampled 200,000 times on each of six captured steps; emitted-token counts conditional on visiting a node are compared with the supplied row and, separately, with the

autoregressive row reconstructed under the native tie-keeping top- k , using an approximate chi-square G -test with pooled rare bins, at least 2,000 visits per node, and a Bonferroni-adjusted per-step threshold of 10^{-3} . Five planted faults (temperature applied twice, target rows shifted, fixed residual draw, unconditional acceptance, correlated random draws) establish sensitivity. The comparison rows are reconstructed by applying native-style processing (per-path penalties, tie-keeping top- k) to the same captured tree logits; they are not independent native forwards. The audit therefore establishes correct penalty histories and acceptance behavior conditional on the supplied logits; how far those logits are from a native forward is bounded separately by the continuation test (Section 4.3), which compares next-token logits against native processes at every cycle. Two known differences from single-row decoding remain outside the audit: the tree path ignores min- p , and its top- k retains exactly k tokens where native decoding keeps boundary ties.

4.5 Output-length comparability

If two arms sample the same target distribution, identical prompts must yield the same output-length distribution. For each replay corpus we compare, request by request, the mean output length over three tree runs with the mean over three chain-MTP runs, and report totals, the number of requests on which the tree arm is longer, and response-cap hits. This check can detect a large aggregate length shift between arms on fixed prompts; it cannot establish equal output distributions, and it cannot attribute a difference observed in live agent runs, where arms diverge after their first tool call. Table 3 summarizes what each instrument can and cannot establish.

5 RESULTS

5.1 Recorded-request replay

Table 4 reports complete configuration runs on the first corpus, each containing all 43 requests. Across three runs per arm, TreeHost records 28.13 pooled tokens/s and five-token native MTP records 26.12, a 7.7% difference; individual run rates span 27.40–28.78 and 26.04–26.23 tokens/s. Plain autoregressive decoding records 10.54 tokens/s in one run, and five-token MTP has the highest pooled rate of the exploratory native sweep. SGLang EAGLE with seven steps and eight draft tokens, whose top- k of one makes the draft a chain, yields 29.69 pooled tokens/s over three runs with identical incoming token IDs, 5.3% above TreeHost; attention, prefill settings, proposal depth, suffix extension, and endpoint timing all differ, so the gap is not a controlled measurement of any one factor. Output-side counters pooled over the repeated vLLM runs give 4.35 accepted drafts per speculative event for TreeHost and 3.17 for five-token MTP; these counters describe emitted drafts, excluding the correction or bonus token. Exploratory variants that disable individual TreeHost optimizations, with their phase timers and drafting-pass sensitivity, are in Appendix E; they predate the penalty-history correction and do not isolate single effects.

On the disjoint corpus (Table 5), TreeHost records 30.76 pooled tokens/s against native MTP-5’s 27.52, an 11.8% difference, and every individual tree rate exceeds every MTP-5 rate. SGLang records 30.49 pooled over three runs (30.24–30.88), 0.9% below TreeHost, with a different chat serialization. The second corpus supports

the direction of the MTP-5 difference without establishing statistical significance, equal output distributions, or task-completion improvement.

The cost of hosting tree verification is measured within one engine (Table 6): on identical prompts the 32-row tree step takes 189.0 and 184.8 ms against 159.0 and 154.3 ms for the six-row chain step, 19–20% more, and accepts 5.32 and 5.68 tokens per step against 4.15 and 4.25, 28–34% more; the pooled-rate advantage is the net of the two. Step times divide pooled decode seconds by speculative events. The native arm has no GPU phase timers, but the native sweep gives a cost curve: plain decoding takes 94.9 ms per step with one verification row, and chain MTP with one, three, five, and seven drafts takes 122.5, 143.3, 159.0, and 180.7 ms, about 9.7 ms per additional drafter pass and verification row. TreeHost runs the same five drafter passes (its drafter span is 50 ms), so against MTP-5 its extra 30 ms decomposes into about 21 ms of device acceptance and native replay (the dispatch span) and about 9 ms for verifying 32 rather than six rows (target span 112 ms), with suffix lookups on the host. The stock-attention variant (Appendix E) takes 204 ms per step at the same acceptance, which is the 15 ms the grouped split-K kernel recovers. These attributions use the tree arm’s timers and the native curve, not native phase timers.

5.2 Coding-agent study

The coding-agent study completes 30 attempts, one per task and arm (Table 7; task-level outcomes in Appendix B). TreeHost resolves five tasks with eight nonempty patches, of which three fail tests, and two empty patches; native MTP-5 resolves six with two failing and two empty; SGLang resolves five with three failing and two empty. Empty patches count as failures without invoking the evaluator. The live trajectories diverge after the first tool call: TreeHost emits 258,889 output tokens over 248 requests, against 134,643 over 224 for MTP-5 and 158,827 over 210 for SGLang, so its lower pooled decode rate and longer summed agent duration here contrast with the fixed-request replays. The replay check in Section 5.6 finds no large aggregate length shift between the arms on fixed prompts; the cause of the live token-volume difference remains unresolved. The study is descriptive: one attempt per arm, ten selected tasks, no plain-decoding agent control, and no benchmark-wide quality estimate.

5.3 Acceptance and execution spans

The task study emits an average of 4.21 accepted drafts per TreeHost speculative event. This output-side counter excludes the correction or bonus token and does not identify the branch published into persistent state. Physical draft slots include padding and are distinct from logical candidates.

Table 8 characterizes the complete TreeHost cycle on these workloads. Dispatch combines sampling and native replay; the GPU spans do not form an isolated critical-path decomposition. Acceptance and phase durations vary with the request corpus, so higher replay acceptance is not evidence of better task quality.

5.4 Component numerical checks

We test GDN verification and accepted recurrent-state publication on two calibration and two held-out synthetic operand sets. Each

Table 3: The measurement and audit instruments, what each establishes, and what it leaves open.

Instrument	Establishes	Leaves open
Pooled decode rate, Eq. 2, on frozen replay corpora	Serving-rate differences between arms on identical requests, with run-to-run spread; per-step time and tokens per step within one engine	Task-completion speed; a controlled cross-engine comparison (stacks, kernels, proposal depth, continuations all differ); batch sizes above one
Disjoint-task corpus	Whether a difference tuned on one corpus reappears on unseen tasks	Statistical significance; chat-level rather than token-level input parity for SGLang
Component checks against a float64 recurrence	Verifier and committer error relative to the native chain on the deployed geometry, with planted controls	Behavior inside the full model; attention and convolution interplay
Frozen native-envelope rule with planted faults	Full-model continuation error within twice the native spread on every surface and cycle; detectability of five fault types	Bitwise or strict greedy equality; live acceptance loop; cache admission and eviction; 18 of all path combinations
Captured-row and Monte-Carlo sampler audit	Penalty histories on 2,480 rows; walk exactness on supplied rows; agreement with native-style processing of the same logits up to the top- k tie convention; fault sensitivity	The native model’s own distribution (bounded only by the continuation test); universal exactness; min- p ; batch sizes above one
Identical-prompt output lengths	Whether a large aggregate length shift exists between arms on fixed prompts	Equal output distributions; the cause of live-agent token differences
Live coding-agent study	Outcomes, request counts, token volumes, and pooled rate per arm on ten named tasks	Benchmark-wide quality; isolated serving effects; trajectory variance with one attempt per arm

Table 4: Sampled replay of 43 recorded requests, capped at 1,024 output tokens. Rates pool matched token and time totals across runs; ranges are individual run rates, not confidence intervals. SGLang receives the same input token IDs on a different stack and streaming endpoint.

Configuration	Runs	Tokens/s	Run range
vLLM autoregressive	1	10.54	—
vLLM MTP, 1 token	1	15.37	—
vLLM MTP, 3 tokens	1	22.06	—
vLLM MTP, 5 tokens	3	26.12	26.04–26.23
vLLM MTP, 7 tokens	1	25.72	—
TreeHost	3	28.13	27.40–28.78
SGLang EAGLE, 7/8	3	29.69	28.91–30.90

Table 5: Disjoint 43-request replay. Rates pool matched totals; ranges are individual run rates. Request messages are shared, but SGLang uses a different chat serialization.

Configuration	Runs	Tokens/s	Run range
TreeHost	3	30.76	30.38–31.14
vLLM MTP, 5 tokens	3	27.52	27.41–27.71
SGLang EAGLE, 7/8	3	30.49	30.24–30.88

set supplies 48 recurrent instances with the evaluated tree geometry: 32 physical output rows, including padding, and 28 accepted publication paths. Each block runs twice in each of two fresh processes. These repetitions check reproducibility; they do not increase the number of distinct inputs.

For each head, we compare both implementations with a float64 recurrence. We require both root-mean-square and maximum absolute errors to satisfy

$$E_{\text{tree}} \leq 1.10E_{\text{native}} + 2^{-24}M_{64} + 2^{-149}, \quad (3)$$

where M_{64} is the corresponding root-mean-square or maximum magnitude of the reference. Table 9 reports all enumerated cases passing in both processes. All published recurrent states are bitwise equal to the native chain, while 68 calibration and 54 held-out

Table 6: Same-engine per-step cost on identical prompts, pooled over the runs shown. Tokens per step count accepted drafts plus the correction or bonus token; plain decoding emits one token per step.

Corpus	Arm	Runs	ms/step	Tokens/step	Tokens/s
First	vLLM autoregressive	1	94.9	1.00	10.54
First	vLLM MTP-1	1	122.5	1.88	15.37
First	vLLM MTP-3	1	143.3	3.16	22.06
First	vLLM MTP-5	3	159.0	4.15	26.12
First	vLLM MTP-7	1	180.7	4.65	25.72
First	TreeHost, stock attention	3	204.2	5.48	26.82
First	TreeHost	3	189.0	5.32	28.13
Disjoint	vLLM MTP-5	3	154.3	4.25	27.52
Disjoint	TreeHost	3	184.8	5.68	30.76

Table 7: Ten-task coding-agent study, one attempt per task and arm. Minutes sum agent durations, including tools but excluding server startup and evaluation. Decode rates use server-counter differences in Eq. 2.

Arm	Resolved	Requests	Tokens/s	Minutes
TreeHost	5/10	248	26.12	190.8
Native MTP-5	6/10	224	27.17	99.5
SGLang EAGLE	5/10	210	29.89	103.0

output tensors differ in BF16 rounding. Native and candidate results reproduce bitwise across repeats and processes.

Calibration controls detect sibling substitution, swapped state rings, and stale publication metadata. Poisoning an off-path branch leaves the tested selected-path state unchanged, and a zero-replay control is rejected. These results support the tested GDN scan and recurrent publication mechanism.

The fused selection kernel records zero byte mismatches against the reference argmax and top- k over 1,368 input cases and five block-count settings (6,840 configurations) and over 24 captured-graph replays; planted controls fire. The patched attention kernel gives bit-identical output and log-sum-exp across repeats in sixteen

Table 8: TreeHost acceptance and mean execution spans. Dispatch includes acceptance, path handling, and state commitment. GPU timers divide each duration sum by its own span count; wall time covers the decoding step. Disjoint replay pools three runs.

Metric	Task study	Disjoint replay
Accepted drafts per event	4.21	4.73
Target GPU span (ms)	115.43	109.84
Drafter GPU span (ms)	55.64	48.17
Dispatch GPU span (ms)	20.27	20.99
Step wall time (ms)	198.88	185.16

Table 9: GDN component checks on GB10. Bitwise columns compare with the native chain; the paired rule compares both implementations with a float64 reference. Counts are per block and process.

Operand sets	BF16 output bitwise	FP32 state bitwise	Paired rule passed
Calibration	3,004/3,072	2,688/2,688	5,760/5,760
Held-out	3,018/3,072	2,688/2,688	5,760/5,760

determinism cases with matching digests across two processes; on synthetic operands at model scales, 93.31% of outputs agree with the reference within two ULP, the maximum output difference is 2^{-8} , and log-sum-exp disagreement is at most four ULP, with all nine declared checks passing. Runtime traces confirm the specialized kernel in all sixteen attention layers, fused selection, device acceptance, and captured publication during the workload runs.

5.5 Full-model continuation

All 126 cycles satisfy the frozen rule (Table 10); on every surface the candidate’s maximum error is no larger than a native control’s maximum deviation from the same reference, and all ten planted faults are detected at their mutation cycles, the weakest at 3.19 times its bound and the stale-input fault at 34–50 times on the logit surface. The next-token KL bound is at least 0.30 nats, so the 0.095 maximum reflects the native spread rather than a tight test. The candidate greedy token matches *A* on 121/126 cycles; the five disagreements occur where the reference’s top two logits tie exactly. The separate-process, cache-capacity, and packed-kernel controls agree with *A* on 125, 125, and 123 cycles. Every cache-resumed case starts with at least 91.9% of its prefix cached across all five arms (30 observations), and the repeated candidate case is bitwise identical over its seven cycles. The result establishes agreement under the stated bounds, not bitwise model equality or strict greedy-output equality; harvested paths do not exercise the live acceptance loop, MTP drafter state, batch sizes above one, or cache admission and eviction.

5.6 Sampler audit

All 2,480 captured rows carry the per-path penalty history; 82 coincide with the flattened chain history where the two agree. This

Table 10: Full-model continuation over 126 cycles: maximum deviation from the sequential reference *A* for the candidate and for the three native controls, and the candidate’s maximum per-cell ratio to the native envelope (limit 2). Maxima may occur at different cycles. State errors are relative L_2 ; KL in nats.

Surface	TreeHost	<i>B</i>	<i>V</i>	<i>P</i>	Ratio
GDN recurrent state	0.422	0.434	0.431	0.436	1.09
Convolution history	0.350	0.347	0.315	0.296	1.11
KV, newly committed	0.373	0.479	0.394	0.465	1.34
KV, earlier positions	0.248	0.295	0.246	0.284	1.07
Next-token KL	0.095	0.162	0.096	0.079	0.63

Table 11: Sampler-audit fault sensitivity on six captured steps. Detection depends on which nodes the walk visits.

Injected fault	Steps detected
Temperature applied twice	5/6
Target rows shifted	6/6
Fixed residual draw	4/6
Unconditional acceptance	3/6
Correlated random draws	1/6

capture postdates a correction: an earlier capture of the same deployment found all 2,480 rows flattened, with node-level total variation above 0.01 on about 22% of deeper decisions (maximum 0.421), which the audit exposed and the serving patch repaired; every measurement in this paper uses the corrected sampler except the exploratory variants in Appendix E. The Monte-Carlo test detects no departure in 48 node tests against the supplied rows. Against rows reconstructed from the same logits with native-style processing, one of 48 node tests rejects; across all 1,120 node-steps the reconstructed total variation is at most 0.028, and the three nodes above 0.01, including the rejected one, are exact ties at the top-*k* boundary. All five planted fault types are detected in at least one of six steps (Table 11). Non-rejection supports this finite check and does not prove universal exactness.

The output-length check finds no large aggregate shift: on identical prompts the corrected tree arm and MTP-5 produce 18,916 versus 18,504 mean output tokens over three runs on the first corpus and 15,270 versus 15,453 on the disjoint corpus, with the tree arm longer on 23 and 19 of 43 requests and similar response-cap counts. This does not establish equal output distributions, and the cause of the live agent study’s token-volume difference remains unresolved. An earlier ten-boot output-distribution diagnostic failed its frozen sensitivity requirement and is uninformative; the tree sampler seeds one process-wide generator with zero, so repeated boots with identical request order are not independent sampling replicates.

6 DISCUSSION AND THREATS TO VALIDITY

6.1 Interpretation

The design couples tree geometry, physical layout, and continuation publication: optimizing any one in isolation moves work

elsewhere or invalidates another stage’s mapping, and native replay pays accepted-path update work to return persistent state to the serving representation each iteration. The evidence supports three statements and no more. TreeHost exceeds the same stack’s chain speculation by 7.7% and 11.8% pooled tokens/s on two corpora of fixed requests; within that engine the 32-row tree step costs 19–20% more than the six-row chain step and accepts 28–34% more tokens, which is the measured cost of hosting tree verification. Against SGLang’s chain configuration on a second engine it is 5.3% slower on the token-identical corpus and 0.9% faster on the disjoint corpus; stacks, kernels, proposal depth, suffix extension, and generated continuations all differ, so that comparison is an observation, not a controlled measurement, and tree drafters, which chain verification cannot host, are not evaluated. The verifier and committer stay within twice the native process-to-process spread on every surface, a bound every planted state fault violates; the candidate’s maximum errors are no larger than the native controls’ own (Table 10). Native replay costs about 21 ms of the 189 ms step (dispatch, including sampling) and 4.5 ms of the 17.7 ms isolated component cycle; it buys published recurrent state bitwise equal to the native chain’s and cache resumption through the engine’s own interfaces. The isolated kernel timings in Appendix F (17.69 ms against Weaver’s 3.45 ms) are measured under different numerical policies and rank nothing, but they do show that this design does not win on isolated kernel time. The deployed sampler applies correct penalty histories and is exact for the probabilities it receives up to the top- k tie convention; the audit does not independently establish the native model’s distribution. The coding-agent study is descriptive: arms execute different request and tool trajectories, one attempt per arm cannot separate serving effects from trajectory variance, the identical-prompt check finds no large aggregate length shift, and the cause of the live token-volume difference is unresolved.

6.2 Threats to validity

External. Measurements use one model, one quantization, one GPU, and batch size one; equal memory-fraction settings do not equalize KV capacity, and boot allocation logs are not peak-memory measurements.

Internal. Each arm has three replay repetitions per corpus, and repeated tree boots reuse a process-seeded random sequence, so none of the differences carries a significance claim or a parity claim.

Construct. The pooled estimator weights long requests by duration, subtracting one token approximates the first output interval, stream parsing remains inside the measured duration, and exact token-ID parity holds only for the first corpus’s SGLang comparison. The numerical rule permits native-relative error and margin-bounded greedy disagreement rather than bitwise equality; 18 schedules do not exhaust path combinations. The sampler audit covers 2,480 captured rows and six Monte-Carlo steps against rows reconstructed from the same logits; the ignored min- p and the exact- k top- k convention are known, unaudited differences. Repeated tree boots reuse random sequences, so the output-length check is not a test of independent samples. The exploratory variant measurements in Appendix E isolate only the split- K attention

effect, and the isolated GDN kernel comparisons in Appendix F use distinct numerical conventions and establish no accuracy ranking.

7 CONCLUSION

TreeHost hosts tree speculative verification on a recurrent-hybrid model inside a production engine by scanning paths in parallel and replaying one accepted path into native state through a shared descriptor. Under a frozen measurement and audit methodology, it verifies 27-candidate trees at 28.13 and 30.76 pooled tokens/s on two recorded corpora, 7.7% and 11.8% above same-stack chain speculation, at a measured per-step cost of 19–20% for 28–34% more accepted tokens, and 5.3% below to 0.9% above a tuned chain stack on another engine; 126 held-out continuation cycles stay within twice the native spread, all ten planted faults are detected, and the sampler is exact for the probabilities it receives up to a top- k tie convention. The audit procedures found and led to the repair of a deployed sampler defect; they are reusable for the tree drafters that should follow, with model-specific surfaces and thresholds re-derived.

REFERENCES

- [1] Anonymous. 2026. Artifact for this submission: serving patches, captured rows, reducers, and experiment records. <https://anonymous.4open.science/r/treehost-artifact/>. Anonymized read-only mirror; records, reducers, and the self-contained audit.
- [2] Hector Borobia, Elies Seguí-Mas, and Guillermina Tormo-Carbó. 2026. Component-Aware Self-Speculative Decoding in Hybrid Language Models. arXiv:2605.01106 [cs.CL] <https://arxiv.org/abs/2605.01106>
- [3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318 <https://arxiv.org/abs/2302.01318>
- [4] Jian Chen, Yesheng Liang, and Zhijian Liu. 2026. DFlash: Block Diffusion for Flash Speculative Decoding. arXiv:2602.06036 <https://arxiv.org/abs/2602.06036>
- [5] Zhuoming Chen, Avner May, Ruslan Svirshchevski, Yuhun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable and Robust Speculative Decoding. In *Advances in Neural Information Processing Systems*, Vol. 37. 129531–129563. <https://doi.org/10.52202/079017-4116>
- [6] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=mZn2Xyh9Ec>
- [7] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for long-context inference. <https://pytorch.org/blog/flash-decoding/>. October 13.
- [8] Dao AI Lab and NVIDIA. 2026. ReplaySSM: Cache SSM Inputs Instead of State. <https://dao-lab.ai/blog/2026/replayssm/>. Author technical report; accessed September 26, 2026.
- [9] Sneha Murthy Ghantasala. 2026. TreeWY: Speculative Verification for Gated DeltaNet Hybrids. arXiv:2608.20961 <https://arxiv.org/abs/2608.20961>
- [10] Sneha Murthy Ghantasala. 2026. TreeWY: Tree Speculative Decoding for Hybrid GDN Models. <https://github.com/vllm-project/vllm/issues/54080>. Author implementation RFC; accessed September 26, 2026.
- [11] Horace He. 2025. Defeating Nondeterminism in LLM Inference. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>. Thinking Machines Lab, September 10, 2025.
- [12] Yuxuan Hu, Ke Wang, Xiaokang Zhang, Fanjin Zhang, Cuiping Li, Hong Chen, and Jing Zhang. 2024. SAM Decoding: Speculative Decoding via Suffix Automaton. arXiv:2411.10666 <https://arxiv.org/abs/2411.10666v3>
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626. <https://doi.org/10.1145/3600006.3613165>
- [15] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, 987–999.

- Vol. 202). PMLR, 19274–19286. <https://proceedings.mlr.press/v202/leviathan23a.html>
- [16] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 7421–7432. <https://doi.org/10.18653/v1/2024.emnlp-main.422>
- [17] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. In *Advances in Neural Information Processing Systems*, Vol. 38. 136737–136756. <https://doi.org/10.52202/085713-4562>
- [18] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, 932–949. <https://doi.org/10.1145/3620666.3651335>
- [19] Yuma Oda, Ryan Mathieu, Roman Knyazhitskiy, and Artur Chakhvadze. 2026. Trees from Marginals: Autoregressive drafting with factorized priors. arXiv:2607.06763 <https://arxiv.org/abs/2607.06763> Technical report.
- [20] Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2025. SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications. arXiv:2411.04975 <https://arxiv.org/abs/2411.04975v3>
- [21] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. 2025. Marconi: Prefix Caching for the Era of Hybrid LLMs. arXiv:2411.19379 <https://arxiv.org/abs/2411.19379>
- [22] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In *Proceedings of Machine Learning and Systems*, Vol. 7.
- [23] SGLang Project. 2026. Qwen3-Next SGLang documentation. <https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>.
- [24] SGLang Team. 2025. Hybrid Models Meet SGLang: More than Full Attention. <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>.
- [25] Snowflake Arctic Inference. 2026. Suffix Decoding. <https://arcticinference.readthedocs.io/en/latest/suffix-decoding.html>. Implementation documentation; accessed September 24, 2026.
- [26] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure. arXiv:2406.17276 [cs.CL] <https://arxiv.org/abs/2406.17276>
- [27] Li Wang, Yi Su, Xiabao Wu, Chiran You, Yongchao Liu, Zhan Qiu, Juelu Zhang, Jiajun Zheng, Fangxin Liu, Jie Zhang, Chen Tian, and Chengying Huan. 2026. Bole: Efficient Tree Speculation for Hybrid-Attention Language Models. arXiv:2608.01651 <https://arxiv.org/abs/2608.01651>
- [28] Zhibin Wang, Xuying Han, Zhaoxia Yang, Fuliang Liu, Xue Li, Rong Gu, Sheng Zhong, and Chen Tian. 2026. SpecLA: Efficient Speculative Decoding for Linear-Attention Models. arXiv:2607.16673 <https://arxiv.org/abs/2607.16673v1>
- [29] Yangchao Wu, Yonatan Dukler, Matthew Trager, Alessandro Achille, Wei Xia, and Stefano Soatto. 2024. Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding. In *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, Vol. 262. PMLR, 292–304. <https://proceedings.mlr.press/v262/wu24a.html>
- [30] Yangchao Wu, Zongyue Qin, Alex Wong, and Stefano Soatto. 2025. STree: Speculative Tree Decoding for Hybrid State-Space Models. arXiv:2505.14969 [cs.LG] <https://arxiv.org/abs/2505.14969>
- [31] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=r8H7xhYPwz>
- [32] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. In *Advances in Neural Information Processing Systems*, Vol. 37. 115491–115522. <https://doi.org/10.52202/079017-3668>
- [33] Xiangrui Yang, Cheng Peng, Yunfeng Zhao, Liang Zeng, Ao Hu, Jiawei Yang, Shengze Wang, Jingshan Lv, Xiao Liang, Chen Yang, Jiaqiang Liu, and Yiming Qiu. 2026. OneLA: Scaling Linear-Attention Decoding to Large Beams in Generative Recommendation. arXiv:2609.12399 <https://arxiv.org/abs/2609.12399v1>
- [34] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. 2025. DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=2c7pfOQu9k>
- [35] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*, Vol. 7.
- [36] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez,

- Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. 62557–62583. <https://doi.org/10.52202/079017-2000>
- [37] Linfeng Zhong, Songqiang Xu, Huifeng Wen, Tong Xie, Qingyu Guo, Yuan Wang, and Meng Li. 2025. SpecMamba: Accelerating Mamba Inference on FPGA with Speculative Decoding. arXiv:2509.19873 [cs.AR] <https://arxiv.org/abs/2509.19873>

A SERVING CONFIGURATION AND MEASUREMENT NOTES

Table 12: Serving settings for the three-arm task study. Native MTP and TreeHost share the vLLM build. Exact versions and launch settings accompany the supplement.

Setting	Configuration
Target	Qwen3.8-27B; mixed NVFP4/FP8 weights, BF16 execution
Drafter	Same-checkpoint one-layer BF16 MTP in all arms; no separate EAGLE draft weights
TreeHost proposals	27 drafts plus root, padded to 32 rows; maximum depth 11; five MTP passes and suffix extension
Native proposals	MTP: five drafts; SGLang: seven steps, top- k one, capacity eight drafts
Attention	vLLM: forked FlashAttention-2; TreeHost adds grouping 2 and split-K 4; SGLang: FlashInfer
Scheduling	One active sequence; 4,096 prefill tokens for vLLM, 8,192 for SGLang
Context / memory	131,072 tokens; memory fraction 0.70; actual KV capacities not equalized
Agent / budget	Qwen Code 0.19.4; one task at a time; 9,000 seconds
Response cap	24,000 tokens per request
Sampling	Temperature 0.6; top- p 0.95; top- k 20; min- p 0; presence penalty 1.0

All tree runs use presence penalty 1.0. Tables 7, 4, 5, 8, 10, and 13 use per-path penalty histories (Section 5.6); Tables 14–16 predate that correction. The exact- k top- k convention and ignored min- p remain the differences from single-row decoding. Workload rates use Eq. 2; token counts and latency sums cover the same completed requests. First-output latency, tools, and evaluator time remain distinct from decode duration. GPU phase times divide each timer’s duration sum by its own span count. Output-side acceptance counters describe emitted draft lengths and do not identify the committed tree path. For target probabilities p and proposal probabilities q , rejection sampling combines accepted mass $\min(p, q)$ with residual mass proportional to $(p - q)_+$, which gives the target marginal for the probabilities actually supplied [3, 15]; serving additionally requires that subsequent forwards consume the state of the accepted path and that logits processing constructs that path’s target distribution. The artifact [1] holds the serving patches, captured rows, reducers, and every record behind the tables.

B TASK-LEVEL OUTCOMES

Table 13 retains every attempt in the three-arm task study; the Tree-Host column is the attempt with the corrected sampler; the earlier pre-correction attempt is preserved in the supplement. Agent duration includes model service and tool execution, excluding server startup and subsequent evaluation. Empty-patch outcomes are distinguished from patches that reach the test evaluator.

Table 13: Task-level outcomes. IDs have prefix astropy__astropy-. Each cell gives rounded agent minutes and outcome: R, resolved; F, failed tests; E, empty patch.

Task suffix	TreeHost	MTP-5	SGLang
13977	11 / E	10 / F	21 / F
14096	18 / R	20 / E	10 / E
14182	24 / R	6 / E	1 / E
14309	11 / R	2 / R	2 / R
14365	10 / F	2 / R	3 / R
14369	25 / F	18 / R	16 / F
14508	38 / R	7 / R	7 / R
14539	6 / E	3 / R	17 / R
14598	37 / F	27 / F	20 / F
14995	11 / R	5 / R	5 / R

C FULL-MODEL COMPARISON PROTOCOL

The continuation comparison uses five fresh serving processes: sequential reference A , an independent process B , a native cache-capacity control V , a native packed-GDN-kernel variant P , and TreeHost. The reference uses a 40 GiB KV allocation; V uses the candidate’s observed allocation. Candidate capacity follows its deployed memory-fraction policy. All arms consume matching forced tokens and positions and retain their own published state across subsequent cycles.

Six prefixes from four additional Astropy task conversations are disjoint from the earlier continuation and first replay corpora. A separate harvest records natural tree inputs and accepted paths. Each prefix supplies three seven-cycle schedules: a cold salted prefix with a common starting-state import, an unsalted repeat that primes the cache with the same import, and a cache-resumed case with no import. The last cycle accepts only the root to consume the final pending token; next-forward logits also observe the subsequent pending token. The root-only cycle is included in the 126-cycle total. Candidate-input checks compare every one of the 32 verification rows with the recorded tree.

The frozen numerical rule uses the largest same-cycle native-control distance, floored at 0.02 for state relative L_2 and 0.15 nats for next-token KL. Candidate distance must not exceed twice this envelope. A greedy mismatch is permitted only when A ’s top-two logit margin is within $\max(0.25, 2 \max_{B,V,P} \|\Delta \text{logit}\|_\infty)$ or the native controls are categorically unstable outside that band; at most 10% of cells may be native-unstable. Input parity, complete sealed observations, starting-state read-back, and detection of every injected fault are prerequisites. These allowances make the verdict a numerical agreement test, rather than exact token or logit equality.

Five fault types run on two prefixes each at a selected natural cycle: omitted convolution gathering, omitted recurrent replay, incorrect KV remapping, stale root input, and sibling-state publication. Each must exceed the same base case’s bound on its designated surface; sibling publication may exceed any of its three targeted state surfaces. Faults are reported separately from the 126 clean cycles. Prefix-cache coverage requires at least 90% cached tokens in every cache-resumed observation. One clean candidate case is repeated in the same process. Earlier invalid forcing and insensitive

comparison records remain in the supplement with their original dispositions.

D PATH-PARALLEL SCAN AND REPLAY PROCEDURE

Algorithm 2 Path-parallel scan and accepted-path replay

Input: Candidate tree T and the pre-tree recurrent state for each GDN layer
Output: Recurrent outputs for all candidates; native recurrent state for the accepted path

- 1: **for** each GDN layer reached during the target forward **do**
- 2: Compute and record this layer’s rounded node operands.
- 3: **for** each path group, in dependency order **do**
- 4: **parallel for** each ready path, head, and value tile **do**
- 5: Load the working state tile from the pre-tree state or a saved parent cut state.
- 6: **for** each node on this path, in ancestry order **do**
- 7: Update the working tile with this node’s operands; write its recurrent output.
- 8: Save the tile if a downstream path needs this node’s state.
- 9: **end for**
- 10: **end for**
- 11: **end for**
- 12: **end for**
- 13: Complete the target forward and sampling to obtain the accepted materialized path A .
- 14: **for** each GDN layer **do**
- 15: Start from this layer’s pre-tree recurrent state.
- 16: **for** each node on A , in path order **do**
- 17: Apply the native GDN update using this layer’s recorded operands.
- 18: **end for**
- 19: Publish the resulting recurrent state to the request’s native running row.
- 20: **end for**

E EXPLORATORY REPLAY VARIANTS

These measurements predate the penalty-history correction of Section 4.4 and compare configurations under one sampler; they do not isolate single optimization effects. Additional runs on the first corpus disable individual TreeHost optimizations (Table 14). The stock-attention variant completes all 43 requests in three runs and records 26.82 pooled tokens/s, 6.5% below the deployed configuration measured under the same sampler; disabling fused draft selection or precomputed tap-operand preparation records 29.40 and 29.02 in one run each, within the deployed configuration’s run-to-run spread. Phase timers (Table 15) show the stock-attention variant with a 14.6 ms longer target span and a 15.1 ms longer step than two instrumented deployed repeats; fused selection off adds 0.7 ms to the drafter span; operand-preparation removal has no clear separation. Boot logs report 20.54 GiB of model-loading allocation in every variant and CUDA-graph pools of 1.52–1.59 GiB (deployed), 0.42–0.50 GiB (stock attention), 1.97 GiB (fused selection off), and 1.47 GiB (operand preparation off); deployed KV capacity is 203,776 tokens versus 197,632–198,656 for stock attention. Generated trajectories, acceptance lengths, and KV capacities differ

across variants, so these are not isolated causal effects at identical computation.

Table 14: Exploratory replay variants on the first corpus. Each run contains all 43 requests.

	Runs	Tokens/s	Run range
Disabled optimization			
Grouped split-K tree attention	3	26.82	26.67–27.11
Fused draft top- k selection	1	29.40	—
Precomputed tap-operand preparation	1	29.02	—

Phase timers add a descriptive view of these variants (Table 15). Stock attention has a 14.6 ms longer target span and a 15.1 ms longer step than the two instrumented deployed repeats. Disabling fused selection increases the drafter span by 0.7 ms; operand-preparation removal has no clear measured separation. Request inputs are shared, but generated trajectories, acceptance lengths, and actual KV capacities differ, so these are not isolated causal effects at identical computation.

Boot logs report 20.54 GiB of model-loading allocation in every variant. Reported CUDA-graph pools are 1.52–1.59 GiB for the deployed runs, 0.42–0.50 GiB for stock attention, 1.97 GiB without fused selection, and 1.47 GiB without prepared operands. Deployed KV capacity is 203,776 tokens versus 197,632–198,656 for stock attention. These allocation reports do not measure peak serving memory, and the unequal capacities further limit attribution.

The deployed tree’s maximum depth is eleven, but neural drafting uses only five MTP passes; suffix lookup supplies the deeper tail. Three-pass drafting and a suffix-dependent three/five-pass policy, retaining 32 physical verification rows (Table 16), are drafting-policy variants rather than new tree topologies. Three-pass drafting saves about 18 ms per cycle but reduces accepted drafts per event by about 11%; its 0.9% pooled-rate difference from the deployed runs is within the observed variation, and the conditional policy has a lower pooled rate, so the five-pass configuration is retained.

F RECURRENT AGREEMENT AND COSTS UNDER DISTINCT REFERENCES

We compare recurrent verification and accepted-state publication using held-out synthetic operands spanning 48 recurrent instances and 48 value heads. Verification covers the tree’s 28 active nodes. Three accepted-path publications produce a history of 1, 6, and 11 updates, including Weaver’s accepted replay and TreeWY’s final deferred commit. Each configuration is evaluated twice in one process, with a fresh initial state for each cycle. A numerical cell fixes the recurrent instance, output node or publication, and value head; errors are reduced over the remaining vector or matrix entries.

Each cell must satisfy both root-mean-square and maximum absolute error bounds,

$$E_{\text{method}} \leq 1.10E_{\text{seq}} + 2^{-24}M_{64} + 2^{-149}, \quad (4)$$

where errors are measured against the specified float64 recurrence and M_{64} is its corresponding root-mean-square or maximum magnitude. For TreeHost, the sequential comparator is the native GPU recurrence. Weaver and TreeWY use sequential software comparators that do not emulate every tensor-core rounding operation.

Table 15: Mean execution spans of exploratory replay variants, in milliseconds. Each phase pools its own duration and timer count. Deployed uses two instrumented runs; stock attention three; other variants one.

Configuration	Target	Draft	Dispatch	Step
Deployed	112.14	49.84	20.81	189.35
Stock attention	126.78	49.94	21.13	204.44
Fused selection off	112.32	50.57	21.08	190.38
Operand preparation off	111.85	49.85	20.75	188.97

Table 16: Exploratory drafting-pass variants on the first corpus. The deployed row pools two runs; each variant has one. GPU drafting and step wall durations use their own timer counts.

Policy	Tokens/s	Accepted drafts/event	Draft ms	Step ms
Five passes	28.69	4.45	49.84	189.35
Three passes	28.94	3.97	32.01	171.27
Conditional three/five	27.05	4.09	47.71	187.55

Table 17: Recurrent agreement on GB10 under distinct sequential references. Counts give cells outside the paired error bounds among 64,512 output cells and 6,912 state cells per configuration, with identical counts in both repeats. The two reference groups are not a common accuracy ranking.

Configuration	Output cells	State cells
<i>Native GPU recurrence reference</i>		
TreeHost	0	0
<i>Sequential software references</i>		
Weaver author-default	53,745	1,154
Weaver (FP32 preparation)	57,707	388
TreeWY author-default	37,528	6,912

TreeWY retains the author implementation’s normalization floor of 10^{-12} and BF16 dot products. Weaver uses TF32 verification, FP32 elementwise accepted replay, and an additive normalization constant of 10^{-6} . We report both its author-default preparation and a variant with FP32 preparation. The two Weaver variants retain their respective preprocessing conventions.

Table 17 reports the cells outside each implementation’s paired bounds. All configurations produce finite tensors and reproduce bitwise between repeats. TreeWY’s BF16 compact commit and its sequential comparator’s FP32 outer-product updates, for example, are different floating-point computations. The normalization and arithmetic conventions are part of the numerical comparison and must be interpreted alongside its reference choice (Section 6.2). We measure verification and accepted-state publication on shared synthetic operands for 48 GDN instances with the model’s head dimensions, using the 28 active nodes of the deployed 32-row tree. Each measurement has 20 warmup iterations and 100 timed iterations. CUDA

Table 19: Named persistent buffer allocations and numerical error for the GDN measurements. Output and state columns are maximum absolute errors against the float64 recurrence; state uses the deepest accepted path. Buffer totals exclude outputs, separate page pools, graph pools, and temporary per-call allocations; the naive state banks include reserved null and durable slots. These are not peak system memory or measured memory traffic.

Method	Buffers (MiB)	Output error	State error
TreeHost	132.5	4.82×10^{-4}	2.62×10^{-6}
Native path baseline	5,040.0	4.82×10^{-4}	2.62×10^{-6}
PyTorch node baseline	5,040.0	4.82×10^{-4}	6.51×10^{-7}
TreeWY author-default	192.8	6.31×10^{-4}	5.10×10^{-3}
Weaver author-default	25.1	5.28×10^{-4}	5.06×10^{-3}
Weaver (FP32 preparation)	31.1	5.18×10^{-4}	8.55×10^{-7}

Table 18: GDN component latency in milliseconds on GB10. Verify, publish, and total columns give graph-mode medians; eager gives the total median without verifier graph replay. TreeWY fuses publication of the previous frontier with the current verification; its final flush is reported separately. These are timing observations under distinct numerical policies.

Method	Verify	Publish	Total	Eager
TreeHost	12.90	4.49	17.69	17.58
Native path baseline	29.71	2.70	32.41	33.73
PyTorch node baseline	52.32	2.70	55.02	81.04
TreeWY author-default	fused		11.01	11.71
Weaver author-default	2.07	1.38	3.45	5.88
Weaver (FP32 preparation)	4.05	1.40	5.45	9.13

events bracket verification and publication; a 256-MiB buffer is written outside each timed iteration. Graph mode captures the verifier while retaining each method’s publication wrapper; eager mode invokes the verifier directly. Accepted paths cycle through all 28

possible frontiers. Durable state is reset before each measurement and evolves across iterations. These measurements exclude attention, convolution, sampling, model projections, and interleaved execution of the remaining model layers.

The native path baseline uses native recurrent updates with the same path cover but stores every node’s full state. The PyTorch baseline instead updates nodes sequentially in topological order. Both publish by copying the selected leaf into durable state. TreeHost records a 17.69-ms graph-mode total, with a 10th–90th percentile interval of 17.18–17.84 ms, versus 32.41 ms (32.36–32.67) for native paths. Total medians are measured directly; they need not equal sums of region medians.

TreeWY’s steady step takes 11.01 ms. A finite sequence also requires its initial verification without a pending commit and a final deferred publication; the latter takes 10.51 ms in this implementation. Weaver includes accepted-path replay in its total. Its graph-mode topology rebuild takes approximately 0.02 ms separately and is excluded from the table because the topology is reused. These boundaries must be retained in a complete-cycle comparison.

Table 19 sums the tensor allocations explicitly retained for verification and publication. TreeHost’s 96-MiB cut-state buffer is reused across layers; only five cut rows, totaling 15 MiB per layer, are logically exported. This agrees with the storage formula in Section 3.2, but does not measure physical memory transactions. Operand rings and publication staging contribute the remaining named buffers. The deployment’s separately allocated 1.5-GiB recurrent page pool is excluded. The naive baselines retain a 34-slot bank (32 physical nodes, one null slot, and one durable slot) and a staging copy for publication. Thus allocation boundaries differ across implementations. Framework peak allocations and all raw timing samples are preserved in the supplement.

The errors in this experiment are descriptive, rather than an admission criterion. TreeWY and the two Weaver variants retain the preprocessing and precision differences described in Section F. Consequently, faster timings here do not establish equal-accuracy superiority, and they do not overturn the paired-bound failures in Table 17. These isolated GDN measurements also do not predict a full-model decoding speedup.